

CSC 330 Programming Languages

Lecture 7.1: Functional Paradigm - Scheme

Introduction

Let's start with a simple example to help build intuition about this language. Here is a factorial function in Scheme:

```
(define (fact x)
  (if (< x 1)
      1
      (* x (fact (- x 1)))
  )
)

(display (fact 5))
```

For comparison, here is the same function in Python:

<pre>(define (fact x) (if (< x 1) 1 (* x (fact (- x 1))))) (display (fact 5))</pre>		<pre>def fact (x): if (x < 1): return 1 else: return x * fact(x - 1) print(fact(5))</pre>
---	--	---

There are several differences here. The smallest difference is that we use "define" instead of "def" to create a function. The parentheses placement is probably the biggest thing to catch your eye. When creating a function, we put the first paren before the function name. We do the same thing when calling a function.

You may have noticed something off about the use of our comparison and arithmetic operators since $(x < 1)$ becomes $(< x 1)$. But actually, more is going on here than you think. Scheme doesn't really have any operators! $<$, $>$, $=$, and so on aren't operators. Neither are $*$, $+$, $-$, and $/$. All of these are functions! And, as functions, we put the name of the function first, followed by its parameters. Imagine if we did this in Python. This may make more sense:

```
def less(x, y): return x < y
def mult(x, y): return x * y
def sub(x, y): return x - y

def fact (x):
    if (less(x, 1)):
        return 1
    else:
        return mult(x, fact(sub(x, 1)))
```

Now imagine if we were allowed to name functions with symbols:

```
def <(x, y): return x < y
def *(x, y): return x * y
def -(x, y): return x - y

def fact (x):
    if (<(x, 1)):
        return 1
    else:
        return *(x, fact(-(x, 1)))
```

Now we just have to move where the parens are and we essentially have Scheme:

```
def fact (x):
    if (< x 1):
        return 1
    else:
        return (* x (fact (- x 1)))
```

This is actually true of a lot of functional languages. Almost everything is a function. In fact, the word "define" is not really a keyword. It's actually a function itself! It's a function that takes in a name, parameters, and instructions and binds the given instructions to that name. So, the define function is taking in "fact" as the name, "x" as the parameter, and the code as the body of the function. It then binds this function to the name "fact" so you can use it later. This brings up another interesting concept of functional languages: code and data are normally one in the same. It has to be this way to have a function such as 'define'. This 'define' function takes in data such as the name of the function you want to define along with its parameters. But it also takes in raw code to form the body of the function. Imagine a function taking in raw code the same as it takes in a value. Imagine if Python did this:

```
def define(name, param, code):
    ...

define('fact', x,
    if (x < 1):
        return 1
    else:
        return x * fact(x - 1)
)
```

This doesn't work in Python since we can't send code to a function. Well, actually we kind of can!

```
define('fact', x, lambda x : 1 if (x < 1) else x * fact(x - 1))
```

Using a lambda function, we can create a function on the fly and send that into another function. A lambda function is just an unnamed, anonymous function. We could then use Python to create this function for us and call it later. Like this:

```
def define(name, body):
    def func(*args):
        return body(*args)
    func.__name__ = name
    globals()[name] = func
    return func

define('fact', lambda x : 1 if (x < 1) else x * fact(x - 1))
print(globals()['fact'].__call__(5))
```

This is essentially what Scheme is doing. We are calling a function (i.e. 'define'), sending it all the information to create our own custom function. It then binds this function to the given name for us and allows us to use it. In fact, we could have written the fact function in Scheme as follows:

```
(define fact (lambda (x)
  (if (< x 1)
      1
      (* x (factorial (- x 1)))
  )
)
```

Lambda functions are a staple of functional languages and Python's lambda functions are borrowed directly from this paradigm. In fact, the ability to set a function directly to a variable is a functional language concept as well. This is called having "first class functions", you can also say that functions are "first class citizens" (both would be correct). In Python, you can actually do this:

```
p = print
p("Hello World")
```

In functional languages such as Scheme, a variable can hold data or code. So, a variable can refer directly to a function. Any language that has this concept built into it, borrowed it from the functional paradigm.

Installation

Before we go any further, let's set up our environment so we can write code in Scheme. To compile and run Scheme code, we are going to use Dr. Racket. This is an editor that's been around for a long time and supports multiple dialects of the Scheme language. You can download Dr. Racket here: <https://download.racket-lang.org>. After installation, fire up the program and follow along with the rest of the notes. Be sure to put:

```
#lang racket
```

At the top of your file so the editor knows which dialect of Scheme you are using. Different dialects have slight differences and without this line, a built-in function might not run correctly. To give you an idea, R5RS, R6RS, R7RS, Guile, PLT Scheme, Racket and Lisp are all dialects of the Scheme language (well, technically Scheme itself is a dialect of Lisp). All of these have small differences. For example, a built-in function may have different names or not be included at all in some dialects.

Variables

Now that you have an idea of what Scheme's syntax looks like, let's go through the basics.

First, there are no actual "variables" in Scheme. In fact, in most functional languages, the concept of a variable is not the same as in imperative languages. A "variable" in a functional language is really more of a mapping or binding. It maps/binds a name to a value. Think of it more like a constant, where the value cannot change once it is assigned. To create a mapping in Scheme, we use the `define` function (yes, the same function that we use to also create our own functions):

```
(define myNum 3)
```

Scheme supports several data types, including ints, reals (which are floats), complex numbers, booleans, strings, and characters.

```
(define pi 3.14)           ; real
(define myComplexNumber 2+5i) ; complex
(define amIUsingScheme #t)  ; boolean
(define myName "Kevin")     ; string
(define letter #\a)         ; character
```

These mappings are now global, meaning they can be used anywhere. If you want to create mappings to be used in a local context, you can create them as parameters to a function, or use the "let" function like so:

```
(let (
  (x 4)
  (y 8)
)

  ; x and y can be used here
  (display (+ x y))
)

; this will give an error since it is outside of the 'let'
(display (+ x y))
```

This creates mappings for `x` and `y` that can be used locally in the next block. The syntax is as follows:

```
(let ; put mappings in this block below
  (
    ; define mappings here of the form (name value)
    ; separate mappings with whitespace (single space or new line)
  )
  ; put code here that uses the mappings above
)
```

Any attempt to use the mappings outside of this will result in an error since they are local to the code block inside the "let".

Arithmetic

As seen in the factorial function above, Scheme technically doesn't have "operators". Our arithmetic operations are actually functions. Like in other languages, functions are written in prefix form (also known as "polish" form). What this means is that the operation comes first, then the operands. Think about this in the context of a normal function call in Python or Java:

```
foo(1, 2)
```

The operation "foo" comes first followed by the operands 1 and 2. Since operators are actually functions in Scheme, they follow this style as well:

```
(+ 1 2)
```

This invokes the + function on the arguments 1 and 2. Scheme has all the basic arithmetic functions built-in:

```
(+ 1 2) ; gives 3  
(- 5 3) ; gives 2  
(- 3 5) ; gives -2  
(* 8 9) ; gives 72  
(* -8 9) ; gives -72
```

We can also send a single argument to these functions to have unary operators:

```
(+ 9) ; gives positive 9 (also known as "affirmation")  
(- 9) ; gives negative 9 (also known as "negation")
```

In fact, we can give these functions any number of arguments and it will carry out the math:

```
(+ 1 2 3 4 5 6) ; gives 21  
(- 6 5 4 3) ; gives -6  
(* 2 4 6 8 9) ; gives 3456
```

The number of parameters a function takes in is known as its "arity" in Scheme. In fact, this term is a common term in functional languages. If you call a function and give it the wrong number of arguments, this is known as "arity mismatch". Although for our arithmetic functions, this isn't an issue. In fact, for + and *, we don't have to give it any arguments at all!

```
(+) ; gives 0  
(* ) ; gives 1
```

We also have division, although the results might be a little surprising.

```
(/ 6 2) ; gives 3
```

This first result doesn't seem so bad, but you may ask if Scheme automatically does integer division if given two integers. The answer is no.

```
(/ 6 5) ; gives  $1\frac{1}{5}$ 
```

It does actual division and will give you a real number if fractions are involved in the answer. However, the way it gives you the result is interesting. It gives it to you as an actual fraction. Here is a screenshot of Dr. Racket to show you:

```
1 #lang racket
2
3
4 (/ 6 2) ; gives 3
5 (/ 6 5) ; gives 1 1/5
6
```

Welcome to [DrRacket](#), version 8.9 [cs].
Language: racket, with debugging; memory limit: 1024 MB.

```
3
1 1/5
> |
```

To write this in code, we would use:

```
(+ 6/5 1/2)
```

This means add $\frac{6}{5}$ to $\frac{1}{2}$ which results in $1\frac{7}{10}$. You may ask why Scheme doesn't interpret this as separate operations. Why doesn't it see this as 6 divided by 5 and 1 divided by 2. Well, first off it would be equivalent mathematically so it doesn't matter much, but secondly, if we wanted to divide 6 by 5, we would have to write `(/ 6 5)`. Since we wrote it as `6/5`, there is no function call going on here and Scheme sees this as a numeric literal. Of course, we could also write it using decimal places as well:

```
(+ 1.2 0.5)
```

If we write it like this, Scheme will give us a result of 1.7 instead of $1\frac{7}{10}$. Obviously, these are mathematically equivalent, but it is interesting to note that Scheme is smart enough to give us a decimal representation if we use decimals and a fractional notation if we use fractions.

What about mod, i.e. the % operator? Well, Scheme has two functions for this:

```
(modulo 8 3) ; gives 2
(remainder 8 3) ; gives 2
```

Both of these would return 2 given the arguments above. Why does Scheme have two functions for this? Well, when the mod operator was introduced to you in the 130 series, you probably recall your professor saying that it is the remainder of a division. For positive numbers, this is true. However, it is a little different for negative numbers. Try the following in Python:

```
print(8 % 3) ; gives 2
print(-8 % 3) ; gives 1
```

For the first one, you get 2 as expected. However, for the second one with -8, you actually get 1. Why is this? For the positive 8, we look at the largest multiple of 3 that's less than or equal to 8. That's 6. We then subtract $8 - 6$ to get 2. However, for -8, the largest multiple of 3 that's less than or equal to -8 is -9. Think about that. -9 is less than -8 and it is the largest multiple of 3 where this is true, since -12 would be less than -9. We then subtract $-8 - -9$ to get 1. So, it does follow the same rules as with positive numbers, however with negative

numbers, it's not really the remainder of a division. To be honest though, we don't often use the mod operator with negative numbers so this isn't much of an issue, but it's important to note that there is a difference here.

In Scheme, we are given two functions such that one can follow the normal mod operation shown above (the same one that Python uses), while the other actually gives us the remainder in all cases. So, we get the following:

```
(modulo -8 3) ; gives 1
(remainder -8 3) ; gives -2
```

Comparisons

Scheme includes the normal comparison operators (technically functions) that you would expect out of most programming languages. Recall that to represent a true or false value, Scheme uses #t for true and #f for false:

```
(< 5 9) ; checks if 5 < 9, gives #t
(< 9 5) ; checks if 9 < 5, gives #f
(< 9 9) ; checks if 9 < 9, gives #f
(<= 9 9) ; checks if 9 <= 9, gives #t

(> 5 9) ; checks if 5 > 9, gives #f
(> 9 5) ; checks if 9 > 5, gives #t
(> 9 9) ; checks if 9 > 9, gives #f
(>= 9 9) ; checks if 9 >= 9, gives #t

(= 9 9) ; checks if 9 = 9, gives #t
(not (= 9 9)) ; checks if 9 ≠ 9, gives #f
```

We have our basic less than, less than or equal, greater than, and greater than or equal. For equality, we use a single = sign (remember that we use "define" for assignment, so we don't need two equal signs to check for equality). For not equals, we actually don't have a symbol for this in Scheme, so we have to use the "not" function to negate the result of an equality check. A tad inconvenient but not a big deal.

Input / Output

To print information on the screen, we use the display function.

```
(display "Hello World")

(define msg "Hey Scheme!")
(display msg)
```

We can display string literals or mappings. We can also grab values from the user by using the read function.

```
(read)
```

We can save the information from the user into a binding:

```
(display "Enter a number: ")
(define myNumber (read))
```

If we want to group multiple statements, we can use the begin function:

```
(begin
  (display "Enter a number: ") ; read a number from the user
  (define myNumber (read))
  (newline)
  ; convert number to string and display it
  (display (string-append "You entered: " (number->string myNumber)))
)
```

Side Effects

With functional languages, almost everything is a function or expression that produces a value. Variables aren't really variables, and as such we can't change their value (well, we kind of can but I'll cover that in a bit). This means that side effects aren't really possible. Remember the example of side effects in Python:

```
def changeMyNumber():
    global myNumber
    myNumber = 10

def changeList(myList):
    myList[0] = 42

myNumber = 5
values = [1, 2, 3]

changeMyNumber()
changeList(values)

print(myNumber)
print(values)
```

As you can see, executing the functions `changeMyNumber` and `changeList` causes the state of the program to change. By "state", I mean our data held in variables. If you run the Python code, you will see that `myNumber` has changed to 10 and `values` have changed to [42, 2, 3]. These changes aren't possible in a functional language. Functions can't change the state of the program. All they can do is take in input and produce output. Keep this in mind as we look at other aspects of Scheme and other functional languages.

If Statements

If statements are a little different in Scheme. First, they aren't technically a "statement". Recall that a statement doesn't produce a value. Its job is to alter the state of the program. Well, since we can't alter state, most of our statements in other languages will be expressions in a functional language. Recall that "expression" produces a value. Here is the syntax of an if expression:

```
(if (condition) (true-part) (false-part))
```

This will check the condition and evaluate to either the result of the true-part or the false-part depending on the condition. Here are some examples:

```
;           condition      true-part           false-part
(display (if (< 5 1) "5 is less than 1" "5 is greater than 1"))
(newline)
```

```
(define x 42)
(display (if (= x 42) "x is 42" "x is not 42"))
(newline)

(display (if (< x 0) (+ x 1) (+ x 2)))
```

What about else if? Well, there are 2 ways we can accomplish this. One is with chaining if expressions:

```
(define x 42)
(display (if (< 5 1)
  "5 is less than 1"
  (if (= x 42)
    "5 is greater than 1 and x is 42"
    "5 is greater than 1 but x is not 42")
  )
)
```

The other approach (that's easier to read), is to use the "cond" function. This is kind of like a cross between an if-else chain and a switch statement. We can specify multiple conditions, and the first true condition will get its corresponding value returned. Here is the same code above using a cond:

```
(display (cond ((< 5 1) "5 is less than 1")
  ((= x 42) "5 is greater than 1 and x is 42")
  (else "5 is greater than 1 but x is not 42")
))
```

Lists

As mentioned briefly above, Scheme is a dialect of Lisp. Lisp stands for List Processing. The entire language is built around lists, and it has a lot of functionality for manipulating lists. Scheme does as well, with special functions to grab the head of the list, the rest of the list, construct a new list, append to an existing list, and so on.

First, let's look at the construction of a simple list of numbers:

```
(define myNumbers '(1 2 3 4 5 6))
```

To create a list, we can use the single quote then put our values in parentheses. Notice there are no commas here. Putting a space between each value is sufficient. Lists are heterogeneous in nature, so different data types are allowed:

```
(define stuff '(1.1 "hello" #f 5 5+2i))
```

Technically, the list above is a collection of pairs of items. The list actually looks more like this behind the scenes:

head rest

1.1	"hello" #f 5 5+2i
"hello"	#f 5 5+2i
#f	5 5+2i
5	5+2i

We can grab the head of any list using the car function and the rest of any list using the cdr (pronounced "could-er") function. For example:

```
(car stuff)           ; 1.1
(cdr stuff)           ; '("hello" #f 5 5+2i)
(car (cdr stuff))     ; "hello"
(cdr (cdr (cdr stuff))) ; '(5 5+2i)
```

If we keep going with cdr, we eventually arrive at an empty list:

```
(cdr (cdr (cdr (cdr (cdr stuff))))) ; '()
```

This is because the last pair of any list contains the last item and an empty list as the rest:

car cdr

5	5+2i ()
5+2i	()

We can construct lists manually using the cons function:

```
(cons 1.1 '("hello" #f 5 5+2i))
```

Here I am constructing the same list as we had before but I'm doing it by putting a 1.1 in front of the list containing "hello", #f, 5, and 5+2i. We would construct the entire list from individual pieces by using cons multiple times to create the pairs:

```
(cons 1.1 (cons "hello" (cons #f (cons 5 (cons 5+2i '())))))
```

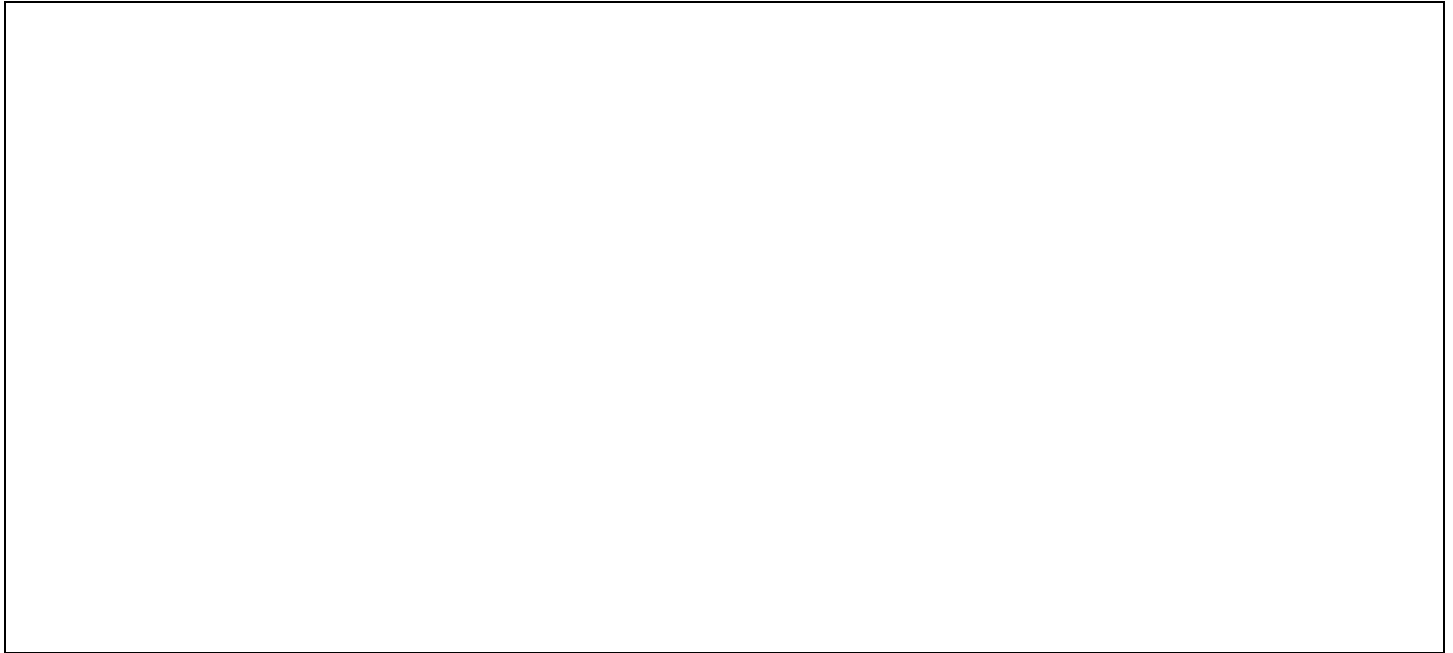
Note the empty list at the end. Remember that lists always end with an empty list. If you create a list of a single element, the car will be that element and the cdr will be an empty list.

What if we want to iterate over a list? Well, we can't really iterate in a functional language, but we can recursively go over it. What if we wanted to sum up all the values in a list? Here is a simple function that can accomplish that:

```
(define (total-value lst)
  (if (empty? lst)
      0
      (+ (car lst) (total-value (cdr lst)))))
(display (total-value '(1 2 3 4 5 6)))
```

This function first checks to see if the list is empty using the "empty?" function (yes, the question mark is part of the function name. Scheme allows functions to have special characters like this). If the list is empty, it returns 0. Otherwise, it adds the first value in the list to the result of a recursive call on the rest of the list. Running this shows the correct answer of 21.

How could I modify this to multiple all items in the list instead? What about if I wanted to add every other item in the list? What about if I only wanted to add numbers that were strictly greater than 2? How about only adding the even values? Take a moment to experiment with lists and try to answer these questions:



What about if the list contains non-numbers? We can do a quick check to see if a value is a number or not before adding it:

```
(define (total-value lst)
  (if (empty? lst)
      0
      (if (number? (car lst))
          (+ (car lst) (total (cdr lst)))
          (total (cdr lst)))
      ))
)

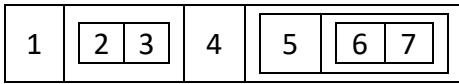
(define myNumbers '(1 "2" 3 4 "5" 6))
(define stuff '(1.1 "hello" #f 5 5+2i))

(display (total-value myNumbers)) ; 14
(display (total-value stuff))      ; 11.1+2.0i
```

We can have nested lists as well. For example:

```
(define nestedNumbers '(1 '(2 3) 4 '(5 '(6 7))))
```

This creates a list of lists which can be better visualized as follows:



How could we alter our function so that it handles this? We could do the following:

```
(define (total lst)
  (if (empty? lst)
      0
      (cond ((number? (car lst)) (+ (car lst) (total (cdr lst))))
            ((list? (car lst)) (+ (total (car lst)) (total (cdr lst))))
            ((total (cdr lst))))
  )
)
```

We first check if the main list is empty and if so, return a zero. If not, we have several checks for the first value in the list. If it's a number, we add it to our total and recursively call the function on the rest of the list. If it's not a number but a sublist, we recursively call the function on both the car and the cdr of the list and add the results. Otherwise, if the car of the list isn't a number nor a sublist, we ignore it and recursively call the function on the cdr of the list.

Of course, we could make this a little more organized by using let:

```
(define (total lst)
  (if (empty? lst)
      0
      (let (
            (head (car lst))
            (rest (cdr lst))
          )
        (cond ((number? head) (+ head (total rest)))
              ((list? head) (+ (total head) (total rest)))
              ( else (total rest) )
            )
      )
  )
)
```

Strings and Symbols

In Java, comparing strings is not straightforward. If you use the double equals operator, Java will compare memory locations. This may evaluate to false even if the two strings have the same characters. Using the "equals" method is preferred as it will actually compare the contents of the strings. The problem, however, is that this comparison is substantially slower as it has to compare each string character-by-character to see if they are a match. Here is an example of what I mean:

```

public class StringCompare {
    public static void main(String[] args) {
        String s1 = "hello";
        String s2 = "hello";

        System.out.println(s1 == s2);           // will result in true
        System.out.println(s1.equals(s2));      // will result in true

        String s3 = "hello";
        String s4 = new String("hello");

        System.out.println(s3 == s4);           // will result in false
        System.out.println(s3.equals(s4));      // will result in true
    }
}

```

For s1 and s2, the java compiler is smart enough to see these as the same string. So, it will optimize memory usage and put them at the same memory location. This means that == and equals() will both return true. However, if the compiler can't figure out the contents of the string during compilation, it won't know to put them at the same memory location. For s4, it sees the "new" keyword and knows that this is dynamic allocation. This is enough to prevent the compiler from seeing s4 the same way it sees s3. So, these will point to different memory locations. This means that s3 == s4 will result in false.

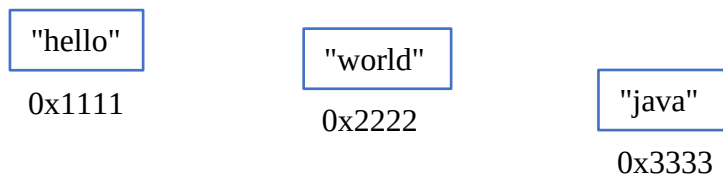
There is a concept at play here. The concept is known as "string interning"¹. The idea is that if the compiler can know ahead of time what the contents of a string is going to be, it can put that string into a set location in memory and place that location into a look up table. For example:

```

// code
String s1 = "hello";
String s2 = "world";
String s3 = "java";

```

Main Memory



```

// intern table

```

"hello"	0x1111
"world"	0x2222
"java"	0x3333

Next time a string is created, this table can be referenced to prevent creating memory that we have already created:

```

// code
String s1 = "hello"; // will refer to memory address 0x1111
String s2 = "world"; // will refer to memory address 0x2222
String s3 = "java";  // will refer to memory address 0x3333
String s4 = "hello"; // will refer to memory address 0x1111

```

This means that if s1 and s4 are checked for equality using ==, it will result in true, since string interning allowed them to be located at the same memory location. This works great if the compiler can figure things out, but if it can't figure out what the contents of the string is going to be, then it must create new memory for

¹ https://en.wikipedia.org/wiki/String_interning

the string even if the contents would have matched something in our table. This means that to properly compare strings and have it result in the correct answer every time, we really should be using the slower "equals()" method, which will compare character-by-character to see if the strings are the same.

What does all this have to do with Scheme? Well, Scheme has a way of forcing this interning to occur. We can use "symbols". Observe the following Scheme code:

```
(define msg1 "hello")
(define msg2 "hello")
(eq? msg1 msg2)      ; checks memory locations
(equal? msg1 msg2)   ; checks character-by-character
```

First, we create two mappings called msg1 and msg2. These will refer to the same string. Since we hard-coded these strings in, Scheme can intern them just like in Java. In Scheme the function "eq?" does a check to see if the two objects occupy the same memory location. Since interning was done, this results in #t. The function "equal?" is different from "eq?". The equal? function checks the content of the memory and so it will do a character-by-character check. It also returns #t. Note that since these are strings, we could have also used the "string=?" function which would do the same as "equal?".

But now, let's confuse the compiler some:

```
(define msg1 "hello")
(define msg2 (string-append "h" "ello"))
(eq? msg1 msg2)
(equal? msg1 msg2)
```

Clearly both msg1 and msg2 are "hello" but the compiler doesn't know this. So, using "eq?" results in #f this time (equal? still results in #t). We can force interning by converting these strings into symbols:

```
(define symbol_msg1 (string->symbol msg1))
(define symbol_msg2 (string->symbol msg2))
(eq? symbol_msg1 symbol_msg2)
(equal? symbol_msg1 symbol_msg2)
```

Now the contents of our strings will be checked against the intern table. If found, we will use the memory location in the table. If not found, we will add the entry to the table then use it. Since the contents of the strings are the same, both symbol_msg1 and symbol_msg2 will refer to the same entry in the table. This means that we can use the faster eq? function and we will get the correct result of #t.

We can also create symbols outright like so:

```
(define symbol1 'Hello)
(define symbol2 'Hello)
(eq? symbol1 symbol2)
```

By putting a single quote before the value, it becomes a symbol instead of a string. Now eq? can be used which will result in a very fast check without having to go character-by-character.

Data Type Checks

We have multiple built in functions for checking the data type of a value or binding:

```
; real
(real? 3)      ; #t
(real? 3.14)   ; #t
(real? 5+2i)   ; #f
(real? 5+0i)   ; #t

; complex
(complex? 5+2i) ; #t
(complex? 5+0i) ; #t

; string
(string? "5")  ; #t
(string? 5)    ; #f
```

Structs

Scheme has some object oriented capabilities, well sort of. Recall that the C language has structs, which are basically just a way of grouping data. It's like having a class with only fields and no methods. Scheme has structs as well. For example:

```
(struct point (x y))
(struct person (name age height))
(struct student (cwid gpa classList))
```

These statements define a 2D point with an x and y; a person with a name, age and height; and a student with a campus wide id, gpa, and list of classes. For comparison, this would be the equivalent in the C language:

```
struct Point { int x; int y; };
struct Person { const char* name; int age; float height; };
struct Student { int cwid; float gpa; const char** classList; };
```

Mostly the same only with {} instead of () and with data types for the fields (also semicolons). Once we create a struct, we can create new instances by just using the struct name:

```
(define myPoint (point 3 5))
(define liam (person "Liam" 8 42.0))
(student 10212123 3.0 (list "class a" "class b"))
```

The use of define is just so we can use these later, but it isn't necessary to create an instance. For example, these instances could be sent to a function so we wouldn't need to use define.

When we create a struct, Scheme automatically gives us some functions for that struct. It gives us a way to determine if an object is of that struct type and a way to retrieve fields within that struct. For example:

```
(point? myPoint) ; returns #t
(point-x myPoint) ; returns 3
(point-y myPoint) ; returns 5
```

The "point?" function that was created when we created the struct will return `#t` if the value we give it is a point object and `#f` otherwise. Functions "point-x" and "point-y" will retrieve the x and y fields respectively. Since we created a person struct, we also have "person?", "person-name", "person-age", and "person-height" available to use. For example, we could do this:

```
(if (person? liam)
    (display (string-append (person-name liam) " is "
                           (number->string (person-age liam)) " years old and "
                           (number->string (person-height liam)) " inches tall"))
    (display "Liam is not a person"))
)
```

This would output:

Liam is 8 years old and 42.0 inches tall

We can also define our own functions for our struct. While we can't technically put these functions into the struct itself, we can name them appropriately. Notice how the functions Scheme gives us all start with the struct name and the retrieval functions have a dash then the name of what they are retrieving. We can follow the same convention. For example, we could make a function for displaying a point:

```
(define (point-display pt)
  (display (string-append "(" (number->string (point-x pt)) ", "
                          (number->string (point-y pt)) ")")))
)
```

We could also do one to find the midpoint between two points:

```
(define (point-mid ptA ptB)
  (point (/ (+ (point-x ptA) (point-x ptB)) 2.0)
        (/ (+ (point-y ptA) (point-y ptB)) 2.0)))
)
```

We would call these functions just like any other:

```
(define pt1 (point 0 1))
(define pt2 (point 2 5))
(point-display pt1)
(point-mid pt1 pt2)
```

We could also make a list of points:

```
(define ptList (list pt1 pt2))
```

Now, for something trickier. Let's design some functionality that will take a list of points, add up all the x's and y's, and return a new point that contains the sum. For example, given the list of points (0, 1), (2, 5), (4, 2), we would return a new point (0+2+4, 1+5+2) which is (6, 8). We will probably need a function to sum all the x values, then a function to sum the y values, then a final function to compose a new point containing the sums:

```
(define (point-sumOfX pts)
  (if (empty? pts)
      0
      (+ (point-x (car pts)) (point-sumOfX (cdr pts)))))
)
```

```

(define (point-sumOfY pts)
  (if (empty? pts)
      0
      (+ (point-y (car pts)) (point-sumOfY (cdr pts)))))

(define (point-sumOfPts pts)
  (point (point-sumOfX pts) (point-sumOfY pts)))

```

We can see a lot of concepts employed here. I strongly encourage you to run the code and see it working for yourself. Then feel free to add your own point functions. For example, how would you create a function that takes in a list of points and returns a new point where its x value is the average of the x values and y is the average of the y values? You might need to use the "length" function to help you out. How about a function that takes a point and a real number and multiplies the point by the number producing a new point (e.g. if given (2, 5) and 6, it would return (2*6, 5*6) which is (12, 30)). How about a function that takes a point and returns its vector length (i.e. its Euclidean distance from the origin)? How about a function that takes two points and returns their Euclidean distance from each other?

To help you out, here is the full code for the point struct functions. Try to get this to run and then try to add the functions talked about above.

```

#lang racket

;; struct
(struct point (x y))

;; functions
(define (point-sumOfX pts)
  (if (empty? pts)
      0
      (+ (point-x (car pts)) (point-sumOfX (cdr pts)))))

(define (point-sumOfY pts)
  (if (empty? pts)
      0
      (+ (point-y (car pts)) (point-sumOfY (cdr pts)))))

(define (point-sumOfPts pts)
  (point (point-sumOfX pts) (point-sumOfY pts)))

(define (point-mid ptA ptB)
  (point (/ (+ (point-x ptA) (point-x ptB)) 2.0)
        (/ (+ (point-y ptA) (point-y ptB)) 2.0)))

(define (point-display pt)
  (display (string-append "(" (number->string (point-x pt)) ", "
                           (number->string (point-y pt)) ")"))
  (newline))

;; instances
(define pt1 (point 0 1))
(define pt2 (point 2 5))
(define pt3 (point 4 2))

```

```

(define ptList (list pt1 pt2 pt3))
; we could have also done this:
; (define ptList (list (point 0 1) (point 2 5) (point 4 2)))

;; main
(point-display (point-sumOfPts ptList))
(point-display (point-mid pt1 pt2))

```

List operations

Most functional languages have a "map", "filter", and "fold" operation. The map operation will go through all items in a list and apply some function to it, resulting in an altered list. For example, given the list '(1 2 3) and a function that squares its input, we would arrive at the list '(1 4 9). Here is how to do that in Scheme:

```

(map (lambda (x) (* x x)) '(1 2 3)) ; returns '(1 4 9)

```

The map function takes a function as its first argument. The function given will take in a value and return the value times itself (i.e. squared). The map function then takes in a list and will apply the given function to all items in the list, resulting in an altered list that it returns.

Like map, filter takes in a function and a list, then returns a list. However instead of applying the function to alter the list, it applies the function to kick items out, returning what is left. For example, you can use filter to retrieve all items that are less than are greater than 2, or all items that are even numbers:

```

(filter (lambda (x) (> x 2)) '(1 2 3 4 5)) ; (3 4 5)
(filter (lambda (x) (= 0 (remainder x 2))) '(1 2 3 4 5 6 7 8)) ; (2 4 6 8)

```

Whereas the map and filter return a list, the fold functions will return a single value. Its objective is to reduce the values in the list to a single value. This could mean adding all the values up, or multiplying them, or finding the min, max, etc. Fold functions take in a function that you want to use, then a starting value, then a list. The following will add all items in the list and return the sum:

```

(foldr (lambda (x y) (+ x y)) 0 '(1 2 3 4))

```

There are two versions of fold, "foldr" and "foldl". Foldr will start at the right side of the list (i.e. the end) and work its way to the front. Foldl will start at the left side of the list (i.e. the front) and work its way to the end. The function you give to foldr or foldl must be able to take in 2 values and return a single value. So, the function I gave takes in two values and returns their sum. The next argument is the starting value to use. The final argument is the list to operate on. Essentially, the above code results in this math expression:

```

(+ 1 (+ 2 (+ 3 (+ 4 0))))

```

Remember that we handle nested parentheses first, so the 4 + 0 will be the first thing to be calculated. Then we will do 3 plus that. Then 2 plus that. Then finally 1 plus that. Resulting in a total of 10. Notice how we went right to left when processing the items in the list. Here are some more examples to help you understand the concept:

```
(foldr (lambda (x y) (/ x y)) 1 '(1 2))  
; Equivalent to: (/ 1 (/ 2 1))  
; Result: 1/2
```

```
(foldl (lambda (x y) (/ x y)) 1 '(1 2))  
; Equivalent to: (/ 2 (/ 1 1))  
; Result: 2
```

```
(foldr cons '() '(1 2 3))  
; Equivalent to: (cons 1 (cons 2 (cons 3 '())))  
; Result: (1 2 3)
```

```
(foldl cons '() '(1 2 3))  
; Equivalent to: (cons 3 (cons 2 (cons 1 '())))  
; Result: (3 2 1)
```

The last example shows a shortcut we can use with the fold functions. Instead of defining our own function to give to fold, we can use a preexisting one. Since functions are first class citizens and are treated similarly to data, we can pass a function's name just like we pass data. So, for constructing a list, we can pass the "cons" function to fold. Since arithmetic also uses functions (instead of actual operators), we can pass those too:

```
(foldr / 1 '(1 2))  
(foldl / 1 '(1 2))  
(foldr + 0 '(1 2 3 4))  
(foldr * 1 '(1 2 3 4))
```

Python has map, filter, and fold (which it calls "reduce") functions available as well. All of these concepts come from the functional paradigm.

Things you shouldn't do but can!

Scheme is a powerful language but sometimes it lets you do things that you really shouldn't. For example, if you run the following code:

```
(+ 5 3)
```

It shouldn't surprise you to get 8 as the output. But what if we run this code instead:

```
(define (+ x y) (- x y))  
  
(+ 5 3)
```

That's right. We can redefine basic functions! This code redefines the + function to actually perform subtraction. Scheme is ok with this and now the line (+ 5 3) actually returns 2! You can do this more succinctly by using:

```
(define + -)
```

It should be obvious why you really shouldn't do this, but feel free to try this out for fun (just don't leave it in your programming assignment!).

As mentioned previously, in functional languages variables really can't change their value. They are treated more like constants. However, in Scheme there is an exception to this rule. Scheme gives you the power to re-map a binding thereby changing its value. Example:

```
(define number 42)
(set! number 43)
(display number)
```

Using the "set!" function, we can remap a previous binding. I wouldn't say this is encouraged to do but it's interesting that Scheme makes it possible. This means we could do things like the following:

```
(define (count lst)
  (if (not (empty? lst))
      (begin
        (set! total (+ total 1))
        (count (cdr lst))
      )
      (newline) ; it has to return something
  )
)

(define total 0)
(count (list 1 3 5 7 9))
(display total) ; displays 5

(set! total 0)
(count (list 10 20 30 40 50 60 70 80 90 100))
(display total) ; displays 10
```

Scheme is still considered to be a functional language (for the most part). However, this is clearly an imperative way of writing a function that counts the number of items in a list. Even though Scheme gives us some leeway with the functional paradigm and allows us, in some cases, to code in more imperative ways, we really shouldn't use this functionality. This "count" function can be rewritten to calculate the size of a list without resetting the value of a variable.

Code Examples

Here are various examples of using Scheme. There are some new functions used in the examples that aren't in the above notes.

; EXAMPLE 1: Converting tabs to commas
#lang racket

```
(define (tabToComma c)
  (if (equal? c #\tab)
      #\,
      c))

(display (list->string (map tabToComma (string->list "1\t2\t3\t4"))))
```

; EXAMPLE 2: Finding the minimum value in a list

```
(define (minWork lst curMin)
  (if (empty? lst)
      curMin
      (if (< (car lst) curMin)
          (minWork (cdr lst) (car lst))
          (minWork (cdr lst) curMin))))

(define (minInList lst)
  (minWork (cdr lst) (car lst)))

(display (string-append "The minimum is "
  (number->string (minInList '(5 8 2 1))) "\n"))

(display (string-append "The minimum is "
  (number->string (minInList '(1 8 2 5))) "\n"))

(display (string-append "The minimum is "
  (number->string (minInList '(5 8 1 2))) "\n"))
```

; EXAMPLE 3: Tower of Hanoi

```
#lang racket

(define (hanoi disks src spr dst)
  (if (< disks 1)
      "Need at least 1 disk"
      (if (= disks 1)
          (string-append src " -> " dst "\n")
          (string-append (hanoi (- disks 1) src dst spr)
                          (hanoi 1 src spr dst)
                          (hanoi (- disks 1) spr src dst)))))

(display (hanoi 3 "A" "B" "C"))
```

; EXAMPLE 4: Binary search

```
#lang racket

(define (search lst target left right)
  (let
    ((mid (truncate (/ (+ left right) 2))))
    (cond
      ((> left right) #f)
      ((= target (list-ref lst mid)) #t)
      ((> target (list-ref lst mid)) (search lst target (+ mid 1) right))
      (else (search lst target left (- mid 1)))
    )
  )
)
```

```

(define (binSearch lst target)
  (search lst target 0 (- (length lst) 1)))

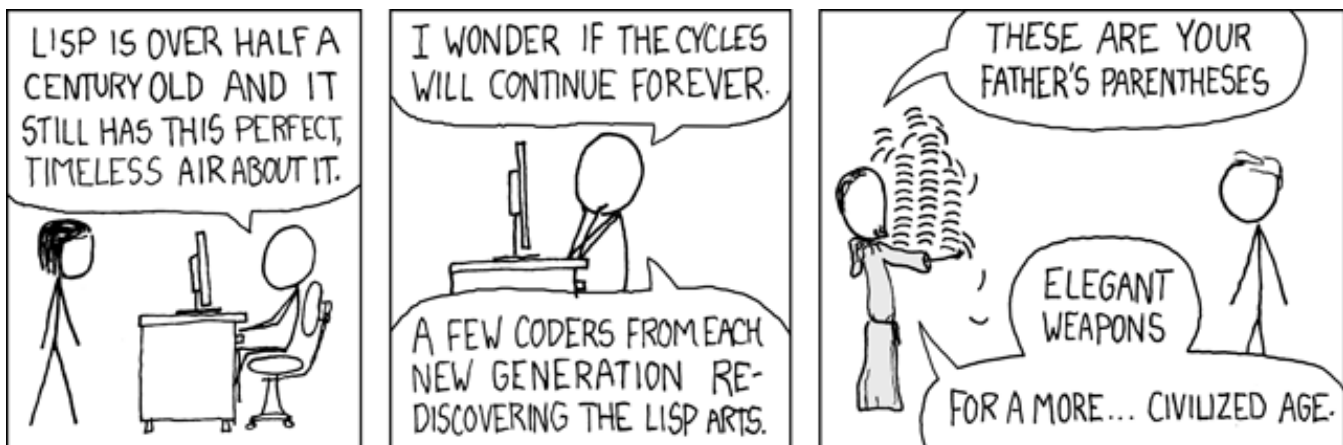
(define lst (sort '(7 3 9 0 2 1 8 6 4 5) <))

(display (binSearch lst 3))
(newline)
(display (binSearch lst 24))
(newline)
(display (binSearch lst -5))

```

Conclusion

By now you should be able to see several benefits to the functional paradigm. First, several non-functional programming languages owe a lot of their features to the functional paradigm. Things like lambda functions, map/filter/fold, functions as first class citizens, lists, recursion, and more all come from the functional paradigm. Since several languages have been inspired by this paradigm, we get these nice features even when using non-functional paradigm languages. So, you can thank languages like Scheme for having the ability to send a function to a function in Python. You should also be able to see how compact some of this code is. For example, operating over lists or calculating math formula can be done in very few lines/characters with a functional language. A lot of the boilerplate code that languages like Java or C require aren't necessary in languages like Scheme. Recursion is also more powerful in functional languages. It's faster to run (if you use "tail recursion") and doesn't have a hard-set requirement for a max number of recursive calls. Ever got a "maximum recursive depth exceeded" message? You won't get that in a functional language. Functional languages are quite powerful and very beneficial to learn, even if you don't get a job directly using them.



References

- https://www.shido.info/lisp/idx_scm_e.html
- <https://docs.racket-lang.org/reference/index.html>
- <https://www.scheme.com/tspl4/>